# ASSIGNMENT-11.2

## NAME: HABEEBA KHANAM

## HALL TICKET :2303A51474

## BATCH NO:29

### LAB 11 – DATA STRUCTURES WITH AI: IMPLEMENTING FUNDAMENTAL STRUCTURES

## TASK DESCRIPTION -1 – (STACK USING AI GUIDANCE)

## CODE:

task -1 (11.1 -tuesday).py | 11.1 - task 2 tuesday.py | 11.1 - task 3 tuesaday.py | 11.1 - task 4 tuesaday.py | task - 5 11.1 (tuesday.py

C: > Users > Sameera Khan > OneDrive > Desktop > task -1 (11.1 -tuesday).py > ...

```python
# simple_stack.py

class Stack:
    """A basic LIFO stack implementation."""

    def __init__(self):
        self._items = []

    def push(self, item):
        """Place an item on top of the stack."""
        self._items.append(item)

    def pop(self):
        """Remove and return the top item.  Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self._items[-1]

    def is_empty(self):
        """Return True if the stack contains no elements."""
        return len(self._items) == 0
```

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
Index 5: []
Index 6: [('job', 'Engineer')]
PS C:\Users\Sameera Khan> & "C:/Users/Sameera Khan/AppData/Local/Programs/P
ython/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/task -1
 (11.1 -tuesday).py"
top: 20
pop: 20
empty? False
size: 1
PS C:\Users\Sameera Khan>
```

Python Debug Console
powershell
Python

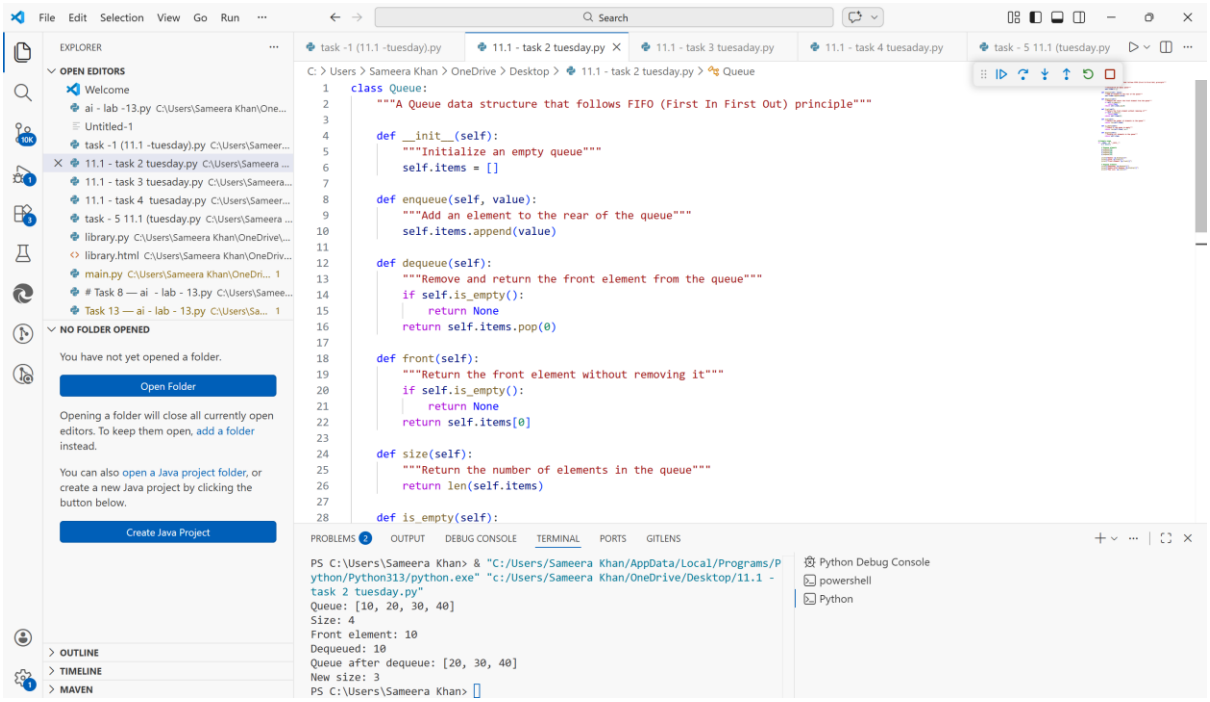
## OBSERVATION:

## 1. STACK IMPLEMENTATION

- The stack followed the **LIFO (Last In First Out)** principle.
- Elements were inserted using the **push ()** operation.
- The **pop ()** operation removed the last inserted element.
- The **peek ()** operation displayed the top element without removing it.
- The program correctly checked whether the stack was empty using **is_ empty ()**.

## TASK DESCRIPTION -2 – (QUEUE DESIGN)

# Code:




# Observation:

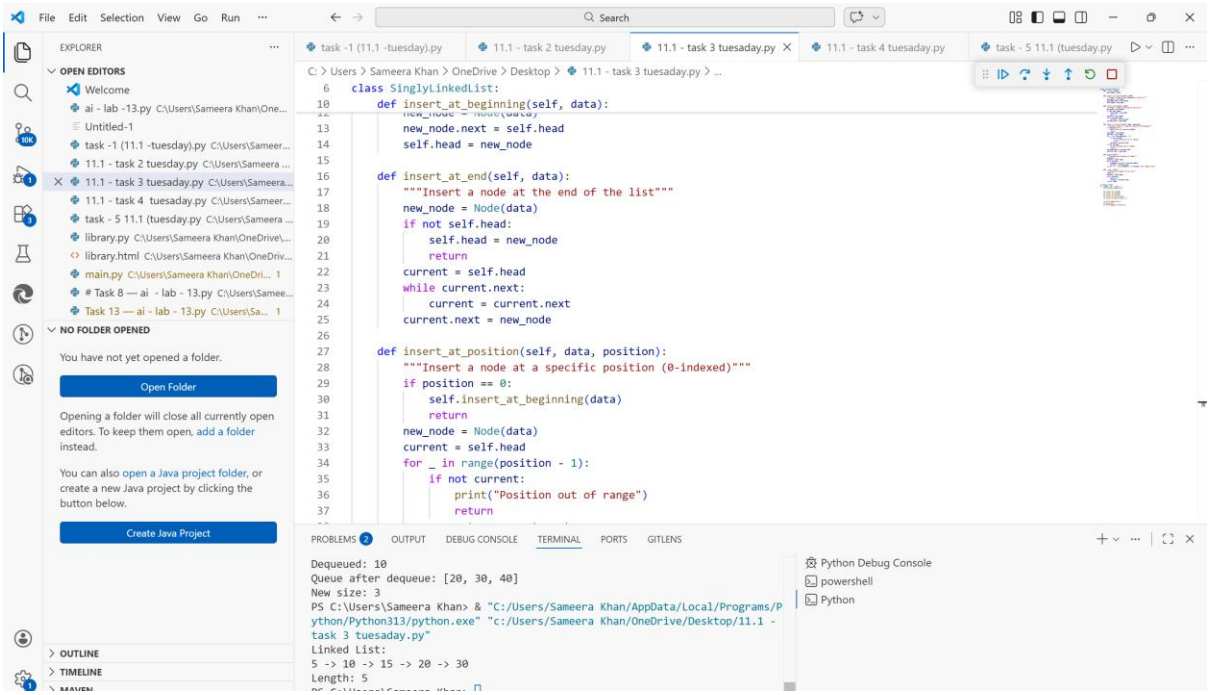
## 1. QUEUE IMPLEMENTATION

- The queue followed the FIFO (First In First Out) principle.
- Elements were inserted using enqueue () and removed using dequeue ().
- The front () function displayed the first element of the queue.

- The size () function correctly calculated the number of elements present in the queue.

## TASK DESCRIPTION -3 –(SINGLY LINKED LIST CONSTRUCTION)

## CODE:



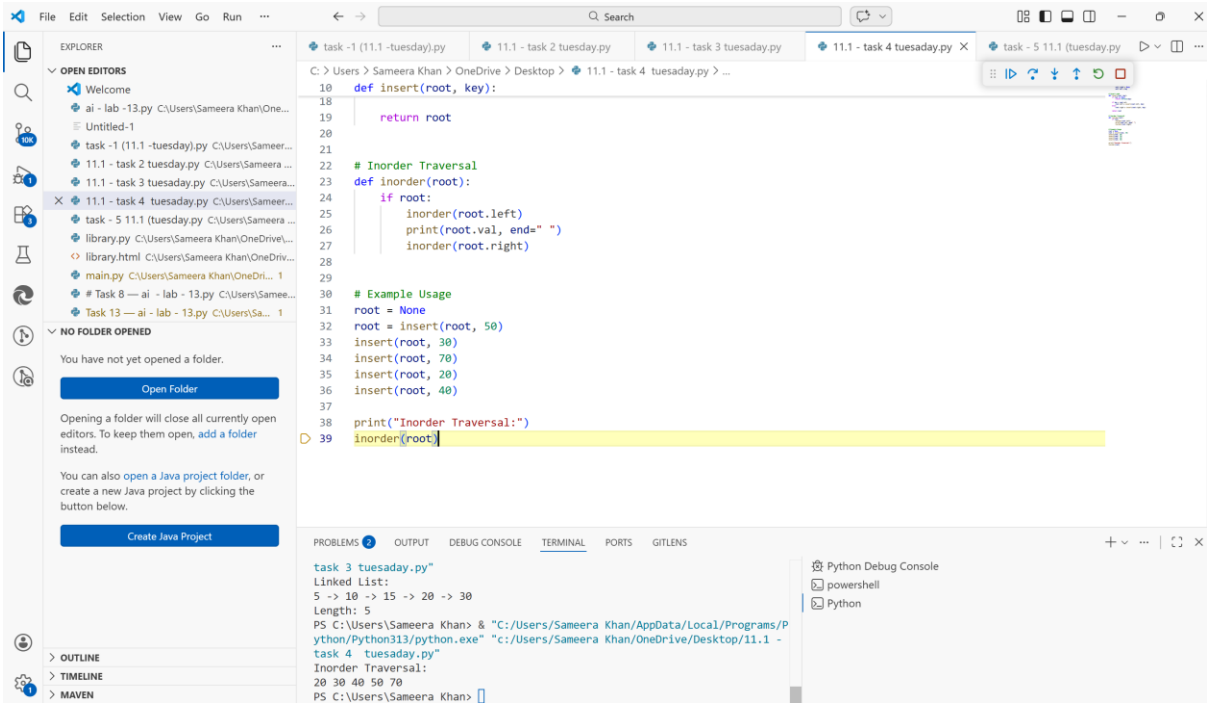

# Observation:

## 1. Singly Linked List

- Nodes were created dynamically using a Node class.
- Each node stored data and a pointer to the next node.
- The insert() method successfully added nodes to the linked list.
- The display () function traversed the list and printed all elements sequentially.

## TASK DESCRIPTION -4 – (BINARY SEARCH TREE OPERATIONS)

## CODE:



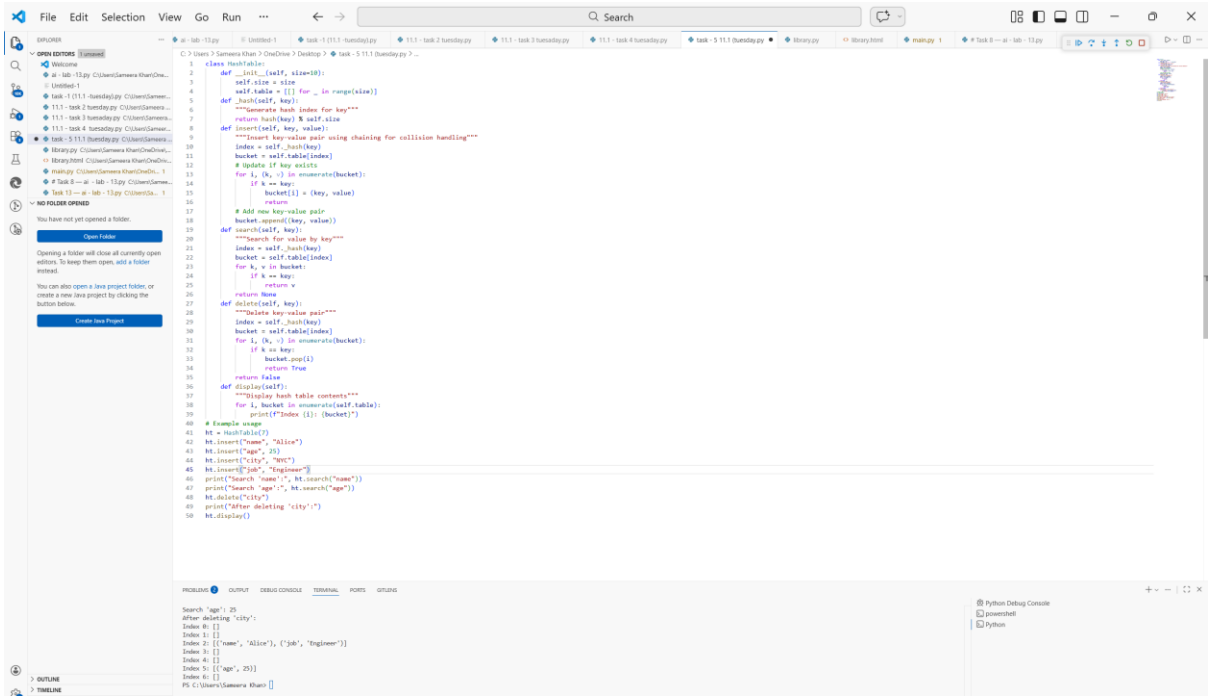

# OBSERVATION:

## 1. BINARY SEARCH TREE (BST)

- The BST stored elements according to the rule Left child < Root < Right child.
- The insert() operation placed elements in the correct position in the tree.
- The in-order traversal displayed elements in sorted order.

# TASK DESCRIPTION -5 – (HASH TABLE IMPLEMENTATION)

# CODE:



```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        """Generate hash index for key"""
        return hash(key) % self.size
    def insert(self, key, value):
        """Insert key-value pair using chaining for collision handling"""
        index = self._hash(key)
        bucket = self.table[index]
        # Update if key exists
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        # Add new key-value pair
        bucket.append((key, value))
    def search(self, key):
        """Search for value by key"""
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None
    def delete(self, key):
        """Delete key-value pair"""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                return True
        return False
    def display(self):
        """Display hash table contents"""
        for i, bucket in enumerate(self.table):
            print(f"Index {i}: {bucket}")
# Example usage
ht = HashTable(7)
ht.insert("name", "Alice")
ht.insert("age", 25)
ht.insert("city", "NYC")
ht.insert("job", "Engineer")
print("Search 'name':", ht.search("name"))
print("Search 'age':", ht.search("age"))
ht.delete("city")
print("After deleting 'city':")
ht.display()
```

```
Search 'age': 25
After deleting 'city':
Index 0: []
Index 1: []
Index 2: [('name', 'Alice'), ('job', 'Engineer')]
Index 3: []
Index 4: []
Index 5: [('age', 25)]
Index 6: []
PS C:\Users\Sameera Khan>
```

# OBSERVATION:

## 1. HASH TABLE

- The hash table stored data using a hash function to determine the index.
- Chaining technique was used to handle collisions.
- The insert(), search(), and delete() operations worked correctly.
- Data retrieval was efficient because of direct indexing.